yield_with_no_args

So far we have seen yield_control, which does not care about arguments, and yield_with_args, which requires certain arguments to be yielded. Sometimes, though, you specifically care that your code yields *no* arguments. For these cases, RSpec offers yield_with_no_args:

11-matchers-included-in-rspec-expectations/03/block_matchers.rb
```
expect { |block| just_yield_these(&block) }.to yield_with_no_args
```

yield_successive_args

Some methods, particularly those in the Enumerable module, can yield many times. To check this behavior, you'd need to combine yield_control's counting ability with yield_with_args's parameter checking.

That's exactly what yield_successive_args does. To use it, you pass one or more arguments, each of which can be an object or a list of objects. The first object or list goes with the first call to yield, and so on:

11-matchers-included-in-rspec-expectations/03/block_matchers.rb
```
expect { |block|
  ['football', 'barstool'].each_with_index(&block)
}.to yield_successive_args(
  [/foo/,                          0],
  [a_string_starting_with('bar'), 1]
)
```

The built-in Ruby function each_with_index will yield twice: first with the two values 'football' and 0, then with the two values 'barstool' and 1. As we did with yield_with_args, we're checking these results using a mix of plain Ruby values, regular expression-style objects, and RSpec matchers.

Mutation

In the wild, it's common for external actions—such as submitting a web form—to change some state inside the system. The change matcher helps you specify these sorts of mutations. Here's the matcher in its most basic form:

11-matchers-included-in-rspec-expectations/03/block_matchers.rb
```
array = [1, 2, 3]
expect { array << 4 }.to change { array.size }
```

The matcher performs the following actions in turn:

1. Run your change block and store the result, array.size, as the *before* value
2. Run the code under test, array << 4